SMART WATER TANK

STM32F446RE (Nucleo-64) + ESP32 DevKit • Ultrasonic Level Sensing + Pump Control + Telegram Alert

■ System Overview

This project implements an IoT-enabled smart water tank monitoring and control system. An **STM32F446RE Nucleo-64** MCU reads water level via an HC-SR04 ultrasonic sensor, drives a relay-controlled pump using hysteresis logic, and transmits live data over UART. An **ESP32 DevKit** receives that data, connects to Wi-Fi, and sends a **Telegram alert** when the tank reaches the full threshold.

Parameter	Value
STM32 Board	Nucleo-F446RE (STM32duino core, Arduino IDE)
ESP32 Board	ESP32 DevKit v1
Level Sensor	HC-SR04 Ultrasonic (trigger PB4, echo PB5)
Tank Height	100 cm (configurable via TANK_HEIGHT_CM)
Sensor Offset	2 cm (configurable via SENSOR_OFFSET_CM)
Pump ON Threshold	≤ 25 % level
Pump OFF Threshold	≥ 90 % level
Full-Tank Alert Level	≥ 88 % (ESP32 side)
UART Baud Rate	9600 bps (STM32 Serial → ESP32 Serial2)
Data Send Interval	2000 ms
Telegram Library	UniversalTelegramBot

■ Hardware Wiring Connections

From	To	Notes
`HC-SR04 GND`	`Common GND`	Shared ground rail
`HC-SR04 TRIG`	`STM32 PB4`	Direct connection, 3.3 V trigger is sufficient
`HC-SR04 ECHO`	`1 kΩ resistor → STM32 PB5`	2 kΩ resistor from PB5 node to GND (voltage divider)

Relay IN	STM32 PB3	Active-HIGH or Active-LOW as per module (check datasheet)
Relay VCC / GND	5 V / Common GND	Powers relay coil driver
Relay COM	Pump supply Live/+	Switched side of pump circuit
Relay NO	Pump Live/+	Normally-open contact drives the pump
STM32 PC6 (TX6)	ESP32 GPIO16 (RX2)	STM32 → ESP32 data link
STM32 PC7 (RX6)	ESP32 GPIO17 (TX2)	ESP32 → STM32 (acks / commands)
STM32 GND	ESP32 GND	Common reference – mandatory for UART
ESP32 5V / VIN	USB or 5 V supply	Powers ESP32 & Wi-Fi radio

■ **Voltage Divider on ECHO pin:** HC-SR04 ECHO outputs 5 V; STM32 GPIO is 3.3 V tolerant only. Use a 1 kΩ series resistor from HC-SR04 ECHO → PB5, then a 2 kΩ resistor from PB5 → GND to divide the voltage to ~3.3 V before it reaches the STM32 pin.

■ STM32F446RE – Firmware Code (copy-paste ready)

Board: Nucleo-F446RE | **Framework:** STM32duino core (Arduino IDE) | **Upload baud:** 9600 | **File:** smart_water_tank_stm32.ino

```
/* ============================================================
   SMART WATER TANK - STM32F446RE (Nucleo-64)
   Function : Ultrasonic level sensing + Relay pump control
              + UART bridge to ESP32 (using Serial only)
   Board    : Nucleo F446RE (Arduino IDE, STM32duino core)
   ============================================================ */

#define TRIG_PIN      PB4
#define ECHO_PIN      PB5
#define RELAY_PIN     PB3
#define STATUS_LED    PA5      // onboard LED (LD2)

// ---- Tank calibration (edit for your tank) ----
const float TANK_HEIGHT_CM   = 100.0;
const float SENSOR_OFFSET_CM = 2.0;
const int   LOW_THRESHOLD    = 25;
const int   HIGH_THRESHOLD   = 90;
const unsigned long SEND_INTERVAL = 2000;

bool pumpState = false;
unsigned long lastSend = 0;

float readDistanceCM() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);
  long duration = pulseIn(ECHO_PIN, HIGH, 30000);
  if (duration == 0) return -1;
  return (duration * 0.0343) / 2.0;
}
```

```
int distanceToLevelPercent(float distanceCM) {
  if (distanceCM < 0) return -1;
  float waterColumn = TANK_HEIGHT_CM - (distanceCM - SENSOR_OFFSET_CM);
  float percent = (waterColumn / TANK_HEIGHT_CM) * 100.0;
  if (percent < 0)   percent = 0;
  if (percent > 100) percent = 100;
  return (int)percent;
}
void setup() {
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(RELAY_PIN, OUTPUT);
  pinMode(STATUS_LED, OUTPUT);
  digitalWrite(RELAY_PIN, LOW);
  Serial.begin(9600);   // used to talk to the ESP32
}
void loop() {
  float distance = readDistanceCM();
  int level = distanceToLevelPercent(distance);
  if (level >= 0) {
    if (!pumpState && level <= LOW_THRESHOLD) {
      pumpState = true;
    } else if (pumpState && level >= HIGH_THRESHOLD) {
      pumpState = false;
    }
    digitalWrite(RELAY_PIN, pumpState ? HIGH : LOW);
    digitalWrite(STATUS_LED, pumpState ? HIGH : LOW);
  }
  if (millis() - lastSend >= SEND_INTERVAL) {
    lastSend = millis();
    Serial.print("L:"); Serial.print(level < 0 ? 0 : level);
    Serial.print(",P:"); Serial.print(pumpState ? 1 : 0);
    Serial.print(",E:"); Serial.println(level < 0 ? 1 : 0);
  }
  delay(300);
}
```

Section	Description
`#define pins`	TRIG_PIN=PB4, ECHO_PIN=PB5, RELAY_PIN=PB3, STATUS_LED=PA5 (onboard LD2)
`Calibration constants`	TANK_HEIGHT_CM, SENSOR_OFFSET_CM, LOW_THRESHOLD (25%), HIGH_THRESHOLD (90%), SEND_INTERVAL (2000 ms)
`readDistanceCM()`	Sends 10 µs TRIG pulse, measures ECHO duration via pulseIn (timeout 30 ms), converts to cm using speed-of-sound formula
`distanceToLevelPercent()`	Converts distance to water level %; clamps to 0–100; returns –1 on sensor error
`Hysteresis logic (loop)`	Pump turns ON when level ≤ 25%, turns OFF when level ≥ 90%. LED mirrors pump state.
`Serial output format`	Every 2 s: L:,P:<0|1>,E:<0|1>\n (Level %, Pump state, Error flag)

■ ESP32 DevKit – Firmware Code (copy-paste ready)

Board: ESP32 DevKit v1 | **Framework:** Arduino (ESP32 Arduino core) | **UART2:** RX=GPIO16, TX=GPIO17 @ 9600 baud | **File:** smart_water_tank_esp32.ino

```
/* ============================================================
   SMART WATER TANK - ESP32 DevKit (Telegram "Tank Full" Alert Only)
   DEBUG VERSION - extra Serial prints to diagnose why the
   Telegram message isn't sending.
   ============================================================ */

#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <UniversalTelegramBot.h>

const char* ssid     = "prasan";
const char* password = "123456789";

#define BOT_TOKEN "8902121825:AAF4KxEF-0kyMeDAuFBLTlaB2URs113uG5Y"
#define CHAT_ID   "8762693342"

WiFiClientSecure secured_client;
UniversalTelegramBot bot(BOT_TOKEN, secured_client);

#define RXD2 16
#define TXD2 17

const int FULL_LEVEL_THRESHOLD = 88;

int  waterLevel    = -1;
bool pumpOn        = false;
bool sensorError   = false;
bool lastPumpState = false;
bool fullAlertSent = false;
bool firstFrameReceived = false;

String rxBuffer = "";

void connectWiFi() {
  if (WiFi.status() == WL_CONNECTED) return;
  Serial.print("Connecting to WiFi");
  WiFi.begin(ssid, password);
  unsigned long start = millis();
  while (WiFi.status() != WL_CONNECTED && millis() - start < 15000) {
    delay(300);
    Serial.print(".");
  }
  if (WiFi.status() == WL_CONNECTED) {
    Serial.println("\nWiFi connected: " + WiFi.localIP().toString());
  } else {
    Serial.println("\nWiFi connect timed out.");
  }
}

void sendTelegram(String msg) {
  Serial.println(">>> sendTelegram() called with: " + msg);
  if (WiFi.status() != WL_CONNECTED) {
    Serial.println(">>> ABORTED: WiFi not connected.");
    return;
```

```
  }
  bool sent = bot.sendMessage(CHAT_ID, msg, "");
  Serial.println(sent ? ">>> Telegram SUCCESS" : ">>> Telegram FAILED (check token/chat id)");
}
bool parseFrame(String frame) {
  int lIdx = frame.indexOf("L:");
  int pIdx = frame.indexOf(",P:");
  int eIdx = frame.indexOf(",E:");
  if (lIdx == -1 || pIdx == -1 || eIdx == -1) {
    Serial.println("PARSE FAILED on frame: [" + frame + "]");
    return false;
  }
  waterLevel  = frame.substring(lIdx + 2, pIdx).toInt();
  pumpOn      = frame.substring(pIdx + 3, eIdx).toInt() == 1;
  sensorError = frame.substring(eIdx + 3).toInt() == 1;
  return true;
}
void checkTankFull() {
  if (sensorError) {
    Serial.println("checkTankFull: skipped, sensorError=true");
    return;
  }
  bool pumpJustTurnedOff = (lastPumpState == true && pumpOn == false);
  // ---- DEBUG: show every value the decision depends on ----
  Serial.print("checkTankFull -> level="); Serial.print(waterLevel);
  Serial.print(" threshold="); Serial.print(FULL_LEVEL_THRESHOLD);
  Serial.print(" lastPump="); Serial.print(lastPumpState);
  Serial.print(" pumpOn="); Serial.print(pumpOn);
  Serial.print(" pumpJustTurnedOff="); Serial.print(pumpJustTurnedOff);
  Serial.print(" fullAlertSent="); Serial.println(fullAlertSent);
  if ((waterLevel >= FULL_LEVEL_THRESHOLD || pumpJustTurnedOff) && !fullAlertSent) {
    String msg = "Tank Full\n";
    msg += "Level: " + String(waterLevel) + "%\n";
    msg += "Pump: " + String(pumpOn ? "ON" : "OFF");
    sendTelegram(msg);
    fullAlertSent = true;
  }
  if (waterLevel < FULL_LEVEL_THRESHOLD - 10) {
    fullAlertSent = false;
  }
  lastPumpState = pumpOn;
}
void setup() {
  Serial.begin(115200);
  Serial2.begin(9600, SERIAL_8N1, RXD2, TXD2);
  secured_client.setInsecure();
  connectWiFi();
}
void loop() {
  if (WiFi.status() != WL_CONNECTED) {
    connectWiFi();
  }
```

```
  while (Serial2.available()) {
    char c = Serial2.read();
    if (c == '\n') {
      Serial.println("RAW FRAME: [" + rxBuffer + "]");
      if (parseFrame(rxBuffer)) {
        Serial.print("Level: "); Serial.print(waterLevel);
        Serial.print("% | Pump: "); Serial.print(pumpOn ? "ON" : "OFF");
        Serial.print(" | Error: "); Serial.println(sensorError);
        if (!firstFrameReceived) {
          lastPumpState = pumpOn;
          firstFrameReceived = true;
        }
        checkTankFull();
      }
      rxBuffer = "";
    } else {
      rxBuffer += c;
    }
  }
}
```

Function / Variable	Description
`ssid / password`	Wi-Fi credentials – change to your network
`BOT_TOKEN / CHAT_ID`	Telegram bot token and destination chat ID
`FULL_LEVEL_THRESHOLD`	88% – Telegram alert fires when level ≥ this value
`connectWiFi()`	Non-blocking Wi-Fi connect with 15 s timeout; called from loop() to auto-reconnect
`sendTelegram(msg)`	Sends message via UniversalTelegramBot; prints SUCCESS/FAILED to Serial monitor
`parseFrame(frame)`	Extracts L:, P:, E: fields from the UART frame string
`checkTankFull()`	Sends alert when level ≥ 88% OR pump just turned OFF; prevents duplicate alerts via fullAlertSent flag; resets flag when level drops below 78%
`firstFrameReceived`	Prevents false 'pump just turned off' trigger on first UART frame after ESP32 boot
`Serial2 (loop)`	Reads UART2 byte-by-byte; builds rxBuffer until '\n', then parses the complete frame

■ UART Data Protocol (STM32 → ESP32)

Every **2000 ms** the STM32 sends a single line terminated by **\n**:

```
L:<level>,P:<pump>,E:<error>\n
Example:  L:73,P:0,E:0
```

Field	Type	Values	Meaning
`L`	int	0 – 100	Water level in percent
`P`	int	0 or 1	Pump relay state (1 = ON)
`E`	int	0 or 1	Sensor error flag (1 = no echo)

■ Required Libraries & Dependencies

Library	Target	Install via	Purpose
`STM32duino core`	STM32	Arduino IDE Boards Manager	Arduino API for STM32 Nucleo
`WiFi.h`	ESP32	Built into ESP32 Arduino core	Wi-Fi connectivity
`WiFiClientSecure.h`	ESP32	Built into ESP32 Arduino core	TLS/HTTPS client for Telegram
`UniversalTelegramBot`	ESP32	Arduino Library Manager	Telegram Bot API wrapper
`ArduinoJson`	ESP32	Arduino Library Manager	JSON parsing (dependency of TelegramBot)

■ Debugging & Troubleshooting

Symptom	Likely Cause	Fix
PARSE FAILED on frame	Noise or partial UART frame	Check common GND; verify baud rate 9600 on both sides
Telegram FAILED	Wrong BOT_TOKEN or CHAT_ID	Re-create bot via @BotFather; get CHAT_ID from @userinfobot
WiFi connect timed out	Wrong SSID/password or out of range	Double-check credentials; move ESP32 closer to router
Level always -1 / E:1	HC-SR04 echo timeout (>30 ms)	Check 5 V supply to sensor; verify voltage divider on ECHO
Pump not switching	Relay module polarity / wiring	Check if relay is Active-HIGH or Active-LOW; swap logic if needed
Duplicate Telegram alerts	fullAlertSent not resetting	Level must drop below 78% to re-arm; normal hysteresis behaviour